Set Type

A set is an unordered collection of elements much like a set in mathematics.

The order of elements is not maintained in the sets. It means the elements may not appear in the same order as they are entered into the set.

A set does not accept duplicate elements.

Sets are unordered so we can not access its element using index.

Sets are represented using curly brackets { }.

data[0] = 10

Ex:-

data = {10,20, 30, “Python”, “Raj”, 40}

data = {10, 20, 30, “Python”, “Raj”, 40, 10, 20}

Set Type

Set-

Method	Description	Example
add(elem)	Adds an element to the set	s.add(10)
update(iterable)	Adds multiple elements	s.update([2, 3])
remove(elem)	Removes elem; raises error if not found	s.remove(3)
discard(elem)	Removes elem; does nothing if not found	s.discard(3)
pop()	Removes & returns an arbitrary element	s.pop()
clear()	Removes all elements	s.clear()
copy()	Shallow copy of the set	new = s.copy()
union(set2) / `	`	Returns union
intersection(set2) / &	Returns intersection	s & t
difference(set2) / -	Elements in s not in t	s - t
symmetric_difference(set2) / ^	Elements in either but not both	s ^ t
issubset(set2)	s is subset of t?	s.issubset(t)

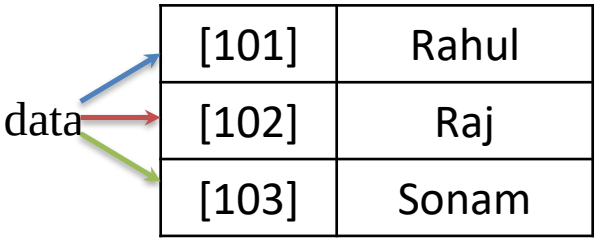
Mapping Type/ dict / Dictionary

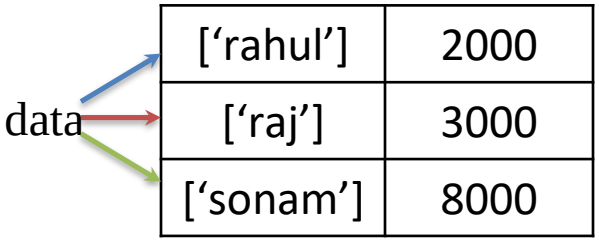
A map represents a group of elements in the form of key value pairs.

Ex:-

data = {101: ‘Rahul’, 102: ‘Raj’, 103: ‘Sonam’ }

data = {‘rahul’:2000, ‘raj’:3000, ‘sonam’:8000, }





Mapping Type/ dict / Dictionary

Dict-

Method	Description	Example
get(key[, default])	Returns the value for key, or default if not found	d.get("age", 0)
keys()	Returns a view of all keys	d.keys()
values()	Returns a view of all values	d.values()
items()	Returns view of key-value pairs	d.items()
update(other_dict)	Merges another dict into this one	d.update({'age': 30})
pop(key[, default])	Removes key and returns its value	d.pop("name")
popitem()	Removes and returns the **last inserted** key-value pair	d.popitem()
setdefault(key[, default])	Returns value if key exists, else sets it	d.setdefault('x', 0)
clear()	Removes all items	d.clear()
copy()	Shallow copy of the dictionary	new = d.copy()
fromkeys(seq[, value])	Creates a new dict from keys with given value	dict.fromkeys(['a', 'b'], 0)

Comparison

Feature / Data Type	String	List	Tuple	Set	Dictionary
Syntax	"abc" or 'abc' or '''abc'''	[1, 2, 3]	(1, 2, 3)	{1, 2, 3}	{"a": 1, "b": 2}
Mutable	✗ No	✓ Yes	✗ No	✓ Yes	✓ Yes
Ordered (Python 3.7+)	✓ Yes	✓ Yes	✓ Yes	✗ No (Unordered)	✓ Yes
Allows Duplicates	✓ Yes	✓ Yes	✓ Yes	✗ No	✗ Keys – No✓ Values – Yes
Indexing	✓ Yes	✓ Yes	✓ Yes	✗ No	✗ No (but keys can be accessed)
Data Types Allowed	Only characters	Any type	Any type	Any immutable type	Keys: immutableValues: any type
Change Elements	✗ No (need new string)	✓ Yes	✗ No	✓ Yes (add/remove)	✓ Yes (update values)
Common Methods	.upper(), .find(), .replace()	.append(), .extend(), .remove()	.count(), .index()	.add(), .remove(), .union()	.keys(), .values(), .items()
Duplicates Handling	Keeps duplicates	Keeps duplicates	Keeps duplicates	Removes duplicates automatically	Keys unique; values can repeat
Performance	Fast for fixed text	Flexible but slower than tuple	Faster than list for fixed data	Fast for membership checks	Fast key-based access
Best Use Case	Text manipulation	Dynamic ordered data	Fixed ordered data	Unique items & fast lookups	Key-value mapping

Operators

An operator is a symbol that performs an operation.

- Arithmetic Operators
- Relational Operators / Comparison Operators
- Logical Operators
- Assignment Operators
- Bitwise Operators
- Membership Operators
- Identity Operators

Arithmetic Operators

Arithmetic Operators are used to perform basic arithmetic operations like addition, subtraction, division etc.

Operators	Meaning	Example	Result
+	Addition	4 + 2	6
–	Subtraction	4 – 2	2
*	Multiplication	4 * 2	8
/	Division	4 / 2	2
%	Modulus operator to get remainder in integer division	5 % 2	1
**	Exponent		25
//	Integer Division/ Floor Division	5//2 -5//2	2 -3

Relational/ Comparison Operators

Relational operators are used to compare the value of operands (expressions) to produce a logical value. A logical value is either True or False.

Operators	Meaning	Example	Result
<	Less than	5<2	False
>	Greater than	5>2	True
<=	Less than or equal to	5<=2	False
>=	Greater than or equal to	5>=2	True
==	Equal to	5==2	False
!=	Not equal to	5!=2	True

Logical Operators

Logical operators are used to connect more relational operations to form a complex expression called logical expression. A value obtained by evaluating a logical expression is always logical, i.e. either True or False.

Operator	Meaning	Example	Result
and	Logical and	(5<2) and (5>3)	False
or	Logical or	(5<2) or (5>3)	True
not	Logical not	not (5<2)	True

Assignment Operators

Assignment operators are used to perform arithmetic operations while assigning a value to a variable.

Operator	Example	Equivalent Expression (m=15)	Result
=	y = a+b	y = 10 + 20	30
+=	m +=10	m = m+10	25
-=	m -=10	m = m-10	5
*=	m *=10	m = m*10	150
/=	m /=10	m = m/10	1.5
%=	m %=10	m = m%10	5
**=	m**=2		225
//=	m//=10	m = m//10	1

Bitwise Operators

Bitwise operators are used to perform operations at binary digit level. These operators are not commonly used and are used only in special applications where optimized use of storage is required.

Operator	Meaning
&	Bitwise AND
|	Bitwise OR
^	Bitwise exclusive OR / Bitwise XOR
~	Bitwise inversion (one's complement)
<<	Shifts the bits to left / Bitwise Left Shift
>>	Shifts the bits to right / Bitwise Right Shift

Bitwise OR |

Operand 1	Operand 2	Result (operand1 | operand2)
True 1	True 1	True 1
True 1	False 0	True 1
False 0	True 1	True 1
False 0	False 0	False 0

a = 10 0 0 0 0 1 0 1 0
b = 15 0 0 0 0 1 1 1 1

Membership Operators

The membership operators are useful to test for membership in a sequence such as string, lists, tuples and dictionaries.

There are two type of Membership operator:-

- in
- not in

```
# String example
'a' in 'apple'     # True

# List example
5 in [1, 2, 3, 4, 5]   # True

# Dictionary example (checks keys)
'key1' in {'key1': 10, 'key2': 20}   # True
```

Identity Operators

The identity operators compare the memory locations of two objects. Hence, it is possible to know whether two objects are same or not.

There are two types of Identity operator:-

- is
- is not

is

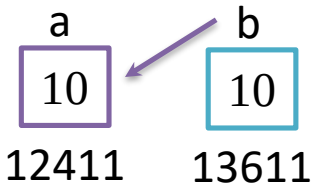
This operator is used to compare whether two objects are same or not.

It returns True if memory location of two objects are same else it returns False.

Ex:-

a = 10

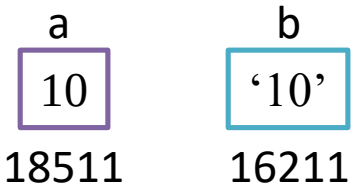
b = 10



a is b Tru
e

a = 10

b = ‘10’



a is b False

is not

This operator works in reverse manner for *is* operator.

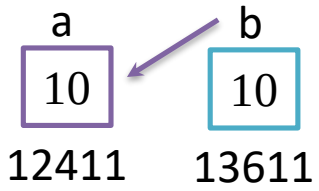
It returns True if memory location of two objects are not same and if they are same it returns False.

Ex:-

a = 10

b = 10

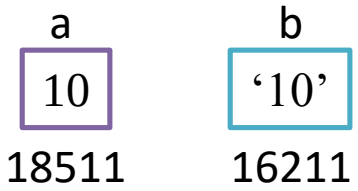
a is not b False



a = 10

b = ‘10’

a is not b True



Operator Precedence and Associativity

General Precedence Order in Python (Highest → Lowest)

1.() Parentheses
2.** Exponentiation (right-to-left)
3.+x, -x, ~x Unary operators
4.*, /, //, %
5.+, -
6.<<, >> Bitwise shift
7.& Bitwise AND
8.^ Bitwise XOR
9.| Bitwise OR
10.Comparisons: <, <=, >, >=, ==, !=
11.not
12.and
13.or
14.= and other assignment operators

Order	Operator	Meaning
1	**( )**	Parentheses
2	****	Exponentiation
3	**+, -, ~**	Unary Plus, Unary Minus, Bitwise Not
4	***, /, //, %**	Multiplication, Division, Floor Division, Modulus
5	**+, -**	Addition, Subtraction
6	<<, >>	Bitwise Left Shift, Bitwise Right Shift
7	**&**	Bitwise AND
8	^	Bitwise XOR
9	>, >=, <, <=, ==, !=	Relational Operators
10	**=, %=, /=, //=, -=, +=, *=, **=**	Assignment Operators
11	is, is not	Identity Operators
12	in, not in	Membership Operators
13	not	Logical NOT
14	or	Logical OR
15	and	Logical AND

- Parentheses
- Exponentiation
- Multiplication, Division, Modulus and Floor Division
- Addition and Subtraction
- Assignment

```
value = (1+1)*2**4//3+4-1
        2*2**4//3+4-1
        2*16//3+4-1
        32//3+4-1
        10+4-1
        14-1
        13
```

Type Conversion

Converting one data type into another data type is called *Type Conversion*.

Type of *Type Conversion*:-

- Implicit Type Conversion
- Explicit Type Conversion

Implicit Type Conversion

In the Implicit type conversion, python automatically converts one data type into another data type.

Ex:-

```
a = 5
b = 2
value = a / b
print(value)
print(type(value))
```

Explicit Type Conversion

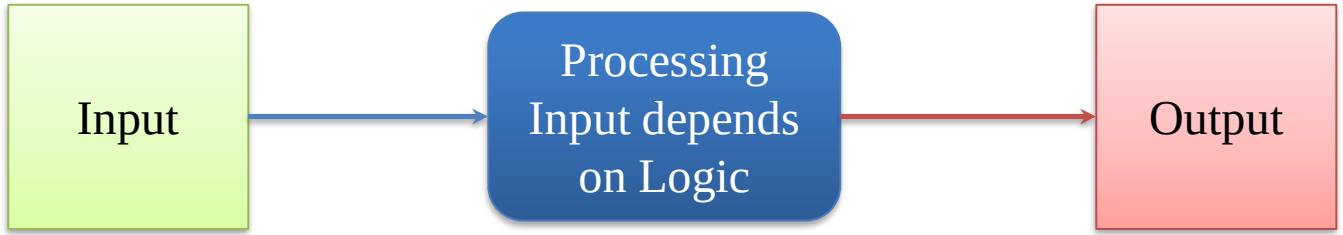
In the Cast/Explicit Type Conversion, Programmer converts one data type into another data type.

- int (n)
- float (n)
- complex (n)
- complex (x, y) where x is real part and y is imaginary part
- str (n)
- list(n)
- tuple(n)
- bin (n)
- oct (n)
- hex (n)

Input and Output

Input - The data given to the computer is called input.

Output – The results returned by the computer are called output.



Keyboard
Mouse
Database

Output Statements

print ( ) Function - The print() function is used to print the specified message to the output screen/device. The message can be a string, or any other object.

Syntax:- print(objects, sep=‘character’, end=‘character’, file=sys.stdout, flush=False)

sep - Separate the objects by given character. Character can be any string. Default is ‘ ’ or can write none.

Output Statements

print(object) - We can pass objects like list, tuples and dictionaries to display the elements of those objects.

Ex:-

data = [10, 20, -50, 21.3, ‘Python’]

print(data)

Input Statements

input( ) – This function is used to accept input from keyboard.

This function will stop the program flow until the user gives an input and end the input with the return key.

Whatever user gives as input, input function convert it into a string. If user enters an integer value still input() function convert it into a string.

So if you need an integer you have to use type conversion.

Syntax:- input([prompt])

prompt is a string or message, representing a default message before input. It is optional

Ex:-

name = input( )

name = input(“Your Name: ”)

mobile = input(“Enter Your Mobile Number: ”)

Input Statements

Whatever user gives as input, input function convert it into a string. If user enters an integer value still input() function convert it into a string.

So if you need an integer you have to use type conversion.

Ex:-

```
mobile = input(“Enter Your Mobile Number: ”)
mb = int(mobile)

mobile = int ( input (“Enter Your Mobile Number: ”) )
price = float ( input (“Total Price: ”) )
mobile = complex ( input (“Enter Complex Number: ”) )
```

Escape Sequence

Escape sequences – Escape sequences are control character used to move the cursor and print characters such as ‘, “. \ and so on.

Escape Sequence	Meaning
\\	Backslash
\’	Single Quote
\”	Double Quote

Escape Sequence	Meaning
\a	Bell
\b	Backspace
\f	Formfeed
\n	NewLine
\r	Carriage Return
\t	Horizontal Tab
\v	Vertical Tab
\newline	Backslash and NewLine Ignored

Comment

Comments are non-executable statements. A Comment is used to describe the feature of a program. Comment helps to understand our program, not only ourselves but also other programmer.

There are two type of programs:-

- Single Line Comment
- Multi line Comment

Single Line Comment

These comments start with a hash symbol (#).

Ex:-

```
# I am single Line Comment
a = 2
b= 32
c = a+b


# This is my first Python Program
# Adding two numbers
```

Multi Line Comment

There is no concept of multi line comment in python but we can create string starting and ending with triple double quotes (”””) or triple single quotes (’’’) which can be used as block of comments.

Since strings are not assigned to any variable, then they are removed from memory by the garbage collector and hence these can be used as comments.

It is not recommended to use triple double quotes or triple single quotes for writing comments as it internally occupy memory and would waste time of the interpreter since the interpreter has to check them.

Ex:-

```
”””
Comment Line 1
Comment Line 2
Comment Line 3
”””
```

```
’’’
Comment Line 1
Comment Line 2
Comment Line 3
’’’
```

If Statement

It is used to execute an instruction or block of instructions only if a condition is fulfilled.

Syntax: -

```
if (condition):
if 5>10:
  a = 1
  b = 2
    statement
Rest of the Code

if (condition):
    statement1
    statement2
Rest of the Code
```

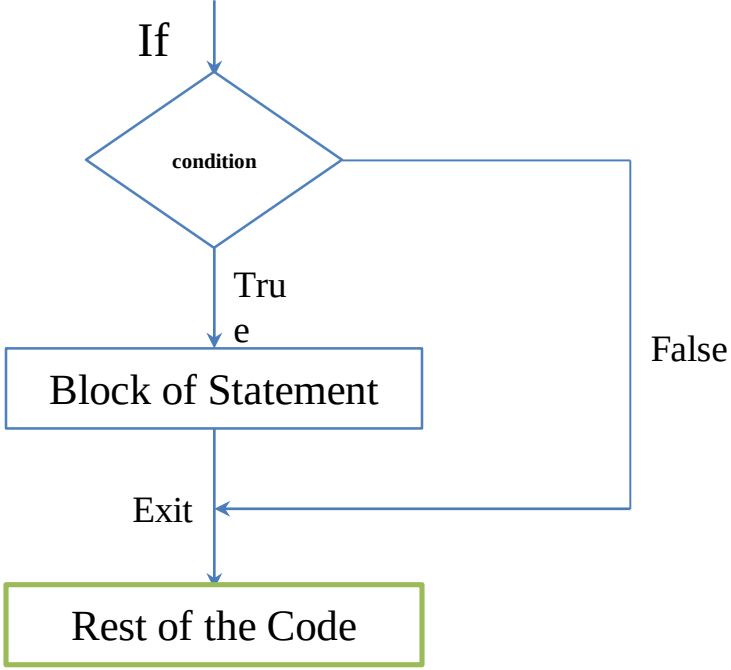
First the condition is tested, If the condition is True

then the statements given after colon (:) are executed

Block of statement/ Group of statements/ Suite

If there is single statement it can be written in one line.

Ex:- if (condition): Statement


If
condition
Tru
e
False
Block of Statement
Exit
Rest of the Code

Nested If Statement

Syntax :

```
if (condition):
    block of statements
    if(condition):
        block of statements
    if(condition):
        block of statements
if(condition):
    block of statements
Rest of the code
```

If Statement with Logical Operator

```
if ( (condition1) and (condition2) ):
        Statement
Rest of the Code

if ( (condition1) and (condition2) ):
        Block of Statements
Rest of the Code

if ( (condition1) or (condition2) ):
        Statement
Rest of the Code
```

Indentation

Indentation refers to spaces that are used in the beginning of a statement. By default python puts 4 spaces but it can be changed by programmers.

```
if (condition):
    Statement
    if(condition):
        statement 1
        statement 2
    if(condition):
        Statement
if(condition):
    Statement 1
    Statement 2
Rest of the code
```

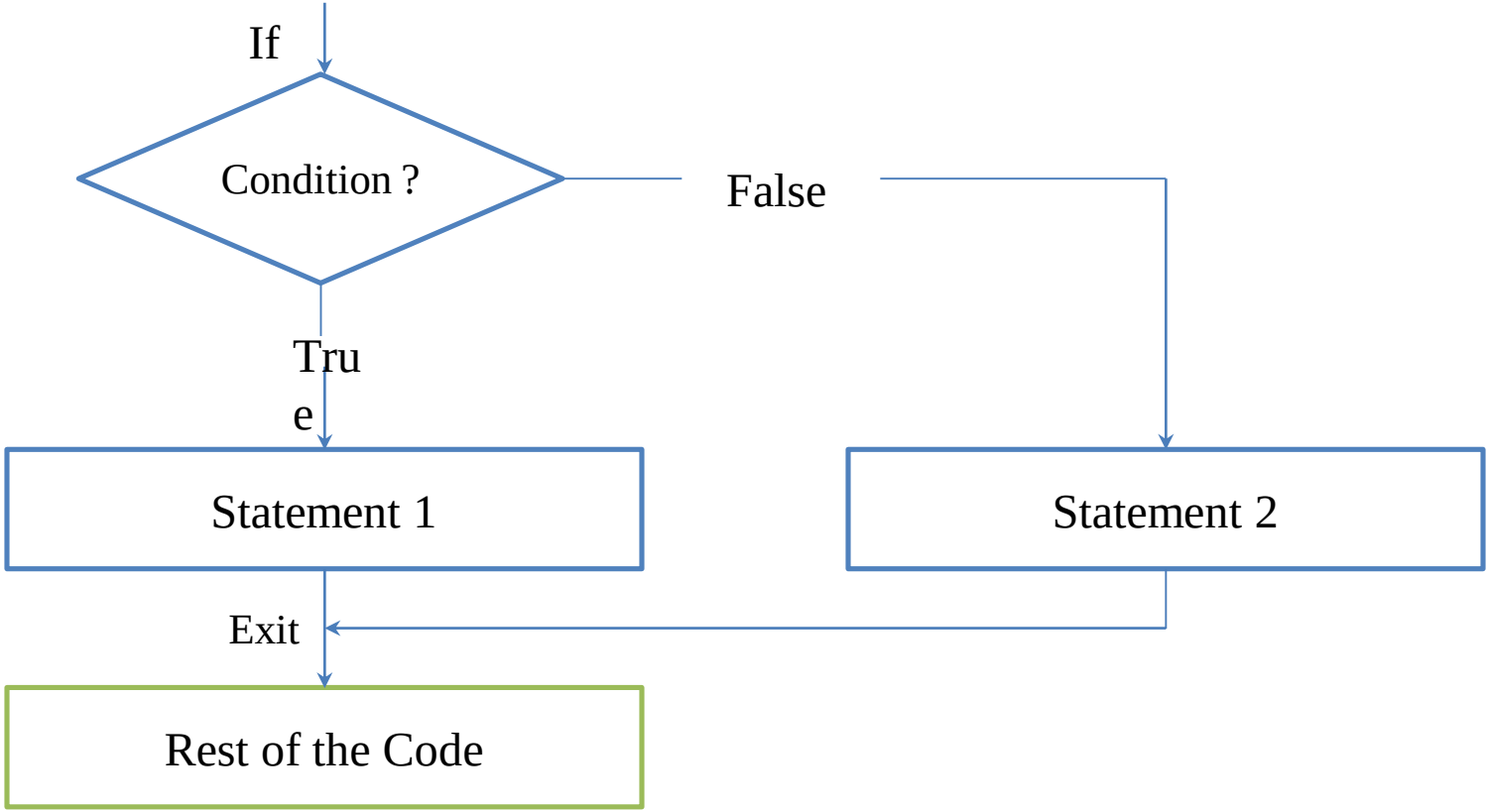
If Else Statement

if… else statement is used when a different sequence of instructions is to be executed depending on the logical value(True/False) of the condition evaluated.

Syntax: -

```
if(condition):
    Statement 1
else:
    Statement 2
Rest of the Code
```

```
if(condition):
    Statement 1
    Statement 2
else:
    Statement 3
    Statement 4
Rest of the Code
```


If
Condition ?
False
Tru
e
Statement 1
Statement 2
Exit
Rest of the Code

Nested If Else Statement

In nested if…. else statement, an entire if… else construct is written within either the body of the if statement or the body of an else statement.

Nested If Else Statement

```
if(condition_1):
        if(condition_2):
                Statement 1
        else:
                Statement 2
else:
        Statement 3
Rest of the Code
```

```
if(condition_1):
        if(condition_2):
                Statement 1
        else:
                Statement 2
else:
        if(condition_3):
                Statement 3
        else:
                Statement 4
Rest of the Code
```

if elif Statement

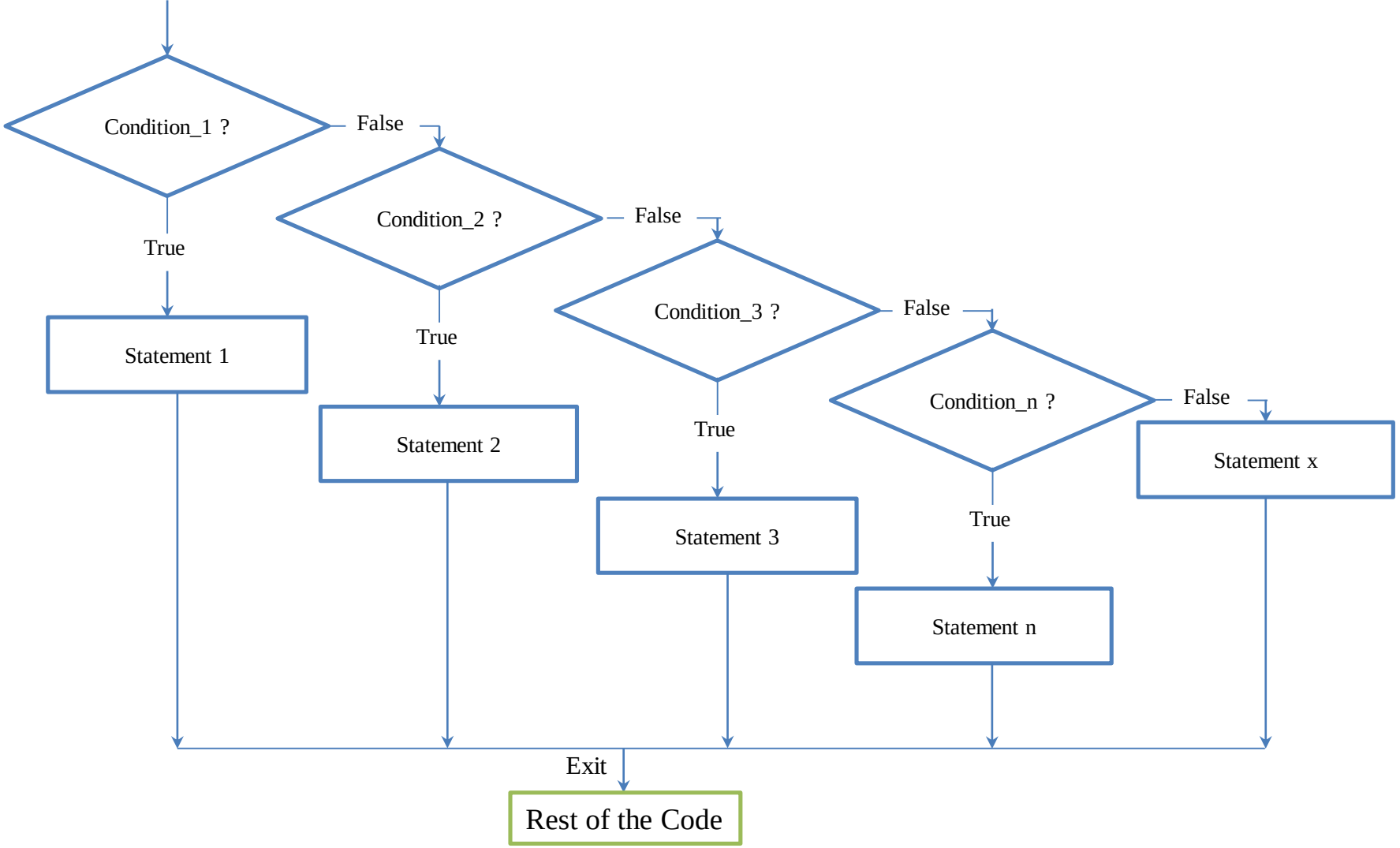
To show a multi-way decision based on several conditions, we use if elif statement.

```
if (condition_1):
        Statement 1
elif (condition_2):
        Statement 2
elif (condition_3):
        Statement 3
elif (condition_n):
       Statement n
Rest of the Code
```

if elif else Statement

To show a multi-way decision based on several conditions, we use if elif else statement.

```
if (condition_1):
        Statement 1
elif (condition_2):
        Statement 2
elif (condition_3):
        Statement 3
elif (condition_n):
        Statement n
else:
        Statements x
Rest of the Code
```


Condition_1 ?
False
True
Statement 1
Condition_2 ?
False
True
Statement 2
Condition_3 ?
False
True
Statement 3
Condition_n ?
False
True
Statement n
Statement x
Exit
Rest of the Code